

0	 h_192.168.170.1	 Firewall-cluster	Any	All			Any	 Management Connection
		 Firewall1						
		 Firewall2						
1	 Firewall2	Any	Any	All			Any	 Allow outbound traffic from Firewalls
	 Firewall1							
2	Any	 Firewall2	TCP  ssh	All			Any	 Stealth Rule - Dropping all the Direct connections
		 Firewall1	TCP  http					
			TCP  https					
			ICMP any ICMP					
3	 h_192.168.170.1	 h_10.1.1.10_ssh-Server	ICMP  any ICMP	All			Any	 SSH Access to the destination server.
			TCP  ssh					
4	Any	Any	Any	All			Any	 Clean-Up Rule

Figure 11: Sample rule-set

connectivity tests, and validate if the packet state is really being failed over to the other peer. For that, we will SSH to the destination server (refer to Figures 1 and 2), and at the same time, we will run *traceroute* to confirm the path.

Once we are certain that traffic is passing through Firewall2, we will bring the *eth2* interface down on Firewall2. This will cause a fail-over, and entire sessions will be transferred over to Firewall1 without disrupting end-to-end connectivity.

Since we are now routing the packets through the firewall, ensure that packet forwarding is enabled on both the firewalls; if not, enable it with the following command:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Here is the *traceroute* output, initiated from 192.168.170.1 to 10.1.1.10, passing through Firewall2 (192.168.170.253):

```
traceroute -n 10.1.1.10
```

```
traceroute to 10.1.1.10 (10.1.1.10), 30 hops max, 60 byte packets
```

```
1  192.168.170.253 1.720 ms 2.750 ms 1.225 ms
2  10.1.1.10      1.225 ms 2.245 ms 2.445 ms
```

Now, let's bring *eth2* down on Firewall2, and notice the hops:

```
[root@Firewall2 ~]# ifdown eth2
```

```
ping 10.1.1.10
```

```
PING 10.1.1.10 (10.1.1.10) 56(84) bytes of data:
64 bytes from 10.1.1.10: icmp_seq=1 ttl=63 time=0.278 ms
64 bytes from 10.1.1.10: icmp_seq=2 ttl=63 time=0.057 ms
64 bytes from 10.1.1.10: icmp_seq=3 ttl=63 time=0.058 ms
```

```
64 bytes from 10.1.1.10: icmp_seq=4 ttl=63 time=0.074 ms
64 bytes from 10.1.1.10: icmp_seq=5 ttl=63 time=0.058 ms
64 bytes from 10.1.1.10: icmp_seq=6 ttl=63 time=0.058 ms
Request timed out.
64 bytes from 10.1.1.10: icmp_seq=8 ttl=63 time=0.278 ms
64 bytes from 10.1.1.10: icmp_seq=9 ttl=63 time=0.057 ms
64 bytes from 10.1.1.10: icmp_seq=10 ttl=63 time=0.058 ms
```

Check the new *traceroute* output; the first hop indicates that traffic is switched over to Firewall1 without any disruptions:

```
traceroute -n 10.1.1.10
```

```
traceroute to 10.1.1.10 (10.1.1.10), 30 hops max, 60 byte packets
1  192.168.170.252 1.750 ms 2.050 ms 2.180 ms
2  10.1.1.10      1.225 ms 1.165 ms 1.115 ms
```

Now, let's see if SSH works without failure, since this is a good medium to validate stateful fail-over. Bring Firewall2's *eth2* interface up again, which will cause the fail-over of services to Firewall2.

I will start an SSH session to the server 10.1.1.10 from 192.168.170.1, and I will then take down *eth2* of Firewall2 again, to see if I can still work in the SSH session:

```
[root@Source_Server1_170.1 ~]# ssh root@10.1.1.10
root@10.1.1.10's password:
```

Once the SSH session has been started, let's bring Firewall2's *eth2* down:

```
[root@Firewall2 ~]# ifdown eth2
```

We check if the SSH session is still alive—and voilà! It is still alive, indicating that stateful fail-over has happened.

If the replication works fine, *conntrackd -s* to display